



Guion de Curi — El proyecto + Catálogo y Reseñas

Tu parte de la presentación de Krypton. Hablás en **5 slides**, en **dos momentos**: presentás el proyecto y la arquitectura (slides 3–5) y después volvés con tus servicios (slides 12–13).

Cómo usar esto: el texto entre «**comillas**» es para LEER en voz alta. Mejor todavía: aprendéte lo y decilo natural. Lo de "En pantalla" es lo que se ve en la diapo (pocas palabras); el detalle lo ponés vos al hablar.

Consejo: el Slide 5 (el diagrama) es el ancla de TODA la charla. Tomate tu tiempo ahí.

Tus turnos: Slides **3, 4, 5** (intro + arquitectura) y **12, 13** (catálogo + reseñas). **Duración:** ~4–5 min.

Slide 3 — El proyecto Krypton

En pantalla: E-commerce B2C de tecnología · Monolito Spring Boot 3 + Java 17 · Frontend React 19 · MySQL 8 · Catálogo, carrito, checkout, órdenes, admin.

Decís:

«Gracias, Tayra. Krypton es una tienda online de tecnología, un e-commerce. Tiene su catálogo de productos, carrito, checkout, las órdenes y un panel de administración. Está hecho con Spring Boot y Java 17 en el backend, React 19 en el frontend y MySQL como base de datos. Y algo importante: esta app YA funcionaba como monolito, con más de 400 tests en verde. O sea, nosotros no reescribimos la lógica; lo que hicimos fue repartirla.»

Slide 4 — ¿Por qué microservicios?

En pantalla: Monolito = todo en un proceso · no escala por partes · un fallo afecta todo · despliegue acoplado. → Microservicios: cada parte independiente, su base, escala y se despliega sola.

Decís:

«¿Y por qué moverla? En un monolito está todo metido en un solo proceso. Eso trae problemas: no podés escalar solo la parte que más se usa, si una parte falla puede caerse todo, y cada vez que querés cambiar algo tenés que desplegar la aplicación entera. Con microservicios, cada parte es independiente: tiene su propia base de datos, escala sola y se despliega sola. Lo que queremos demostrar con esto son cuatro patrones: Eureka para descubrirse, Feign para llamadas directas, RabbitMQ para eventos, y Docker para empaquetar.»

Slide 5 — Arquitectura general

En pantalla: Frontend → **api-gateway (:8080)** → 9 servicios, todos en **eureka (:8761)**, cada uno con su base MySQL, + **RabbitMQ**. Tres ideas: 1 base por servicio · se llaman por NOMBRE · el JWT es el pasaporte.

Decís:

«Y así nos quedó la foto completa. Miren el diagrama: el frontend le habla al api-gateway, en el puerto 8080. El gateway reparte a los nueve servicios, y todos están registrados en Eureka. Cada servicio tiene su propia base MySQL, y además tenemos RabbitMQ para los mensajes. Quédense con tres ideas que se van a repetir en toda la charla: una base de datos por servicio, los servicios se llaman por nombre, y el token JWT es el pasaporte que viaja en cada pedido. A partir de acá, cada uno les va a explicar su parte. Arranca Tayra con la infraestructura.»

👉 **Traspaso:** (el cierre del slide ya pasa la posta a Tayra)

⏸ Pausa — ahora hablan otros

Acá hablan **Tayra** (6–8, infra) y **Jason** (9–11, seguridad). **Tu próximo turno es el Slide 12.** Jason cierra el slide 11 diciendo *"le paso a Curi que les cuenta el catálogo y las reseñas"* — esa es tu entrada.

Slide 12 — catalog-service

En pantalla: Dueño del catálogo (productos, categorías, stock) · base `krypton_catalog` · valida un JWT que NO emitió · expone endpoints internos de stock para el checkout.

Decís:

«Vuelvo yo. Me tocó el catálogo. El catalog-service es el dueño de los productos, las categorías y el stock, con su base propia, `krypton_catalog`. Acá hay un detalle lindo: este servicio NO tiene la tabla de usuarios, pero igual valida el token de cada pedido. ¿Cómo? Con el secreto compartido que les explicó recién Jason. Y además guarda el stock, que es un dato que más adelante el checkout va a necesitar para descontar cuando alguien compra.»

Slide 13 — review-service (reseñas)

En pantalla: Reseñas (rating 1–5 + comentario) · base `krypton_reviews` · **promedio** con `AVG` · valida que el producto exista **vía Feign a catalog** · 1 reseña por usuario por producto.

Decís:

«El otro servicio que armé es el de reseñas. Permite que un usuario le ponga a un producto una calificación del 1 al 5 con un comentario, en su base `krypton_reviews`. Calcula el promedio de estrellas con una agregación, un `AVG`. Y tiene un detalle interesante: antes de aceptar una reseña, le pregunta al catálogo si ese producto realmente existe, y se lo pregunta por Feign. Además, solo deja una reseña por usuario por producto. Reusa todo el patrón: base propia y validación del token.»

👉 **Traspaso:** «Y ahora viene la parte más jugosa: Cristofer les va a contar cómo funciona el checkout cuando tenés que coordinar varios servicios.»

✅ *Listo, terminaste tu parte fuerte. Te quedan la demo y las preguntas finales con el resto.*